

Documentación Técnica — qa-insights

Documentación de referencia del diseño, las decisiones técnicas y el funcionamiento de la herramienta.

Contenido

1. Motivación: el rol de tooling en SDET
2. Test Impact Analysis
3. Detección de flakiness
4. Por qué la herramienta se testea a sí misma
5. Decisiones de diseño
6. Integración en CI
7. Vías de extensión
8. Glosario

1. Motivación: el rol de tooling en SDET

Un automation engineer escribe tests; un SDET, además, **construye la infraestructura que hace al equipo más rápido y efectivo**. Esta herramienta es un ejemplo concreto: en vez de resolver un caso de prueba puntual, ataca dos problemas transversales de cualquier suite que crece —el tiempo de ejecución y la confiabilidad— con soluciones reutilizables por todo el equipo.

2. Test Impact Analysis

El problema

A medida que una suite crece, correrla entera en cada pull request se vuelve lento y caro. Pero correr "algunos tests al azar" pierde cobertura. La solución es correr **exactamente los tests afectados** por los cambios: ni más (lento) ni menos (riesgoso).

Cómo se determina el impacto

1. **Grafo de dependencias** (`dependency-graph.ts`). Se recorren los archivos de código del proyecto y se extraen sus imports relativos mediante análisis estático (sin ejecutar nada). Cada especificador

(`./x`, `../y`) se resuelve a una ruta de archivo concreta, probando extensiones (`.ts`, `.js`, ...) e `index`. El resultado es un grafo `archivo → archivos que importa`.

2. **Cierre transitivo** (`test-impact.ts`). Para cada archivo de test se calcula, con un recorrido en profundidad, el conjunto de **todo** lo que importa en cadena (no solo sus imports directos). Un test está afectado si:

- cambió el propio archivo de test, o
- su cierre transitivo de dependencias intersecta con los archivos cambiados.

El caso transitivo es el que aporta valor real: si `discount.ts` importa `math.ts`, y `discount.spec.ts` importa `discount.ts`, entonces un cambio en `math.ts` afecta a `discount.spec.ts` aunque este no importe `math` directamente. Un análisis ingenuo de un solo nivel se perdería esa relación y podría dejar pasar una regresión.

Origen de los archivos cambiados

La lista de cambios se pasa explícitamente (`--changed`) o se deriva de git (`--since <ref>`), que ejecuta `git diff --name-only`, tal como haría un pipeline al comparar la rama del PR contra `main`.

3. Detección de flakiness

El problema

Los tests inestables (flaky) erosionan la confianza en la suite. Pero hay una distinción crítica que muchos equipos pasan por alto: **un test que falla siempre no es flaky, está roto** — es un bug real o un test obsoleto. Confundirlos lleva a ignorar fallos genuinos como si fueran ruido.

La clasificación

A partir del histórico de N corridas, se agrupan los resultados por nombre de test y se clasifica cada uno:

Clasificación	Condición	Significado
stable	nunca falló	Sano
flaky	pasó y falló al menos una vez	Inestable → prioridad de arreglo
broken	falló siempre	Bug real / test obsoleto (NO flakiness)

Además se calculan métricas de salud de la suite: cantidad y porcentaje de flaky, cantidad de rotos, duración promedio y ranking de los tests más lentos. Estos números convierten una impresión vaga ("la suite anda medio inestable") en datos accionables y priorizables.

La tasa de flakiness

Se define como `fallos / corridas` para cada test. Un test con 40 % de flakiness (falló 2 de 5 corridas) es más urgente que uno con 10 %. El reporte ordena los flaky por esta tasa.

4. Por qué la herramienta se testea a sí misma

Una herramienta de calidad que no está probada es una contradicción. El directorio `tests/` cubre, con Vitest, la lógica central: la extracción de imports, la construcción del grafo, el cálculo de tests afectados (incluido el caso transitivo y el de "sin relación") y la clasificación de flakiness. La lógica está escrita como **funciones puras** justamente para que sea fácil de testear de forma determinista.

Se distingue entre los tests de la herramienta (`tests/*.test.ts` , que sí se ejecutan) y los specs del proyecto de muestra (`sample-project/tests/*.spec.ts` , que son fixtures para el análisis y no se ejecutan). La configuración de Vitest lo separa explícitamente.

5. Decisiones de diseño

- **Sin dependencias de runtime.** La herramienta usa solo la librería estándar de Node. Para un componente de tooling, minimizar la superficie de dependencias reduce el riesgo de cadena de suministro y facilita su adopción.
 - **Análisis estático, no ejecución.** El grafo se construye leyendo el código, sin correrlo. Es rápido, seguro y no requiere un entorno de ejecución del proyecto analizado.
 - **Funciones puras + CLI delgada.** Toda la lógica vive en funciones sin efectos secundarios; el CLI es una capa fina de entrada/salida. Esto hace la lógica testeable y reutilizable como librería.
 - **Formato de resultados normalizado.** El detector consume un formato simple y estable; adaptadores (no incluidos) convertirían el reporter de una herramienta concreta a ese formato.
-

6. Integración en CI

En un pipeline real, ambas capacidades se integran naturalmente:

- **Impact Analysis** en el job de PR: `qa-insights impact --since origin/main` acota qué correr, recortando el tiempo de feedback.
- **Detección de flakiness** en una corrida programada: se acumulan los resultados de las últimas N ejecuciones y se publica el reporte de salud, para priorizar el trabajo de estabilización.

El pipeline de este repositorio ejecuta el type-check, los tests de la herramienta y ambas demos, garantizando que la herramienta funciona de punta a punta.

7. Vías de extensión

- **Aliases de tsconfig:** leer `compilerOptions.paths` para resolver imports con alias (`@pages/...`), además de los relativos.
 - **Adaptadores de reporters:** convertir el JSON de Playwright o JUnit XML al formato de corridas normalizado.
 - **Umbral de cuarentena:** marcar automáticamente como "en cuarentena" los tests que superen cierta tasa de flakiness, para sacarlos del gate bloqueante mientras se arreglan.
 - **Tendencia temporal:** almacenar la salud de la suite en el tiempo para detectar si la flakiness sube o baja entre releases.
 - **Salida en formatos de máquina:** emitir JSON/Markdown para integrarse con dashboards o comentarios de PR.
-

8. Glosario

- **Test Impact Analysis (TIA):** técnica para ejecutar solo los tests afectados por un conjunto de cambios.
- **Grafo de dependencias:** representación de qué archivo importa a qué otro.
- **Cierre transitivo:** todo lo que un archivo importa, de forma directa e indirecta (en cadena).
- **Análisis estático:** examinar el código sin ejecutarlo.
- **Flaky:** test que a veces pasa y a veces falla sin cambios reales.
- **Broken (roto):** test que falla siempre; es un fallo genuino, no flakiness.
- **Tasa de flakiness:** fallos / corridas de un test.
- **Salud de la suite:** conjunto de métricas (flaky %, rotos, duración) que resumen el estado de la suite.
- **Función pura:** función cuyo resultado depende solo de sus entradas, sin efectos secundarios; fácil de testear.